LEARN BEING FORWARD

Programming — TypeScript Programming

Classes & OOP in TypeScript

COMPLETE LEARNING GUIDE

What You'll Learn

TypeScript extends JavaScript's classes with compile-time access control, abstract classes, and interface implementation checks — giving object-oriented code the kind of enforced structure that plain JavaScript classes only achieve through convention and discipline. This sub-module covers class fields and constructors, the public/private/protected access modifiers and the readonly modifier, inheritance with super and method overriding, abstract classes for defining a partial, extensible base, the implements keyword for enforcing an interface's shape on a class, static members, the parameter-property shorthand that collapses constructor boilerplate, getter/setter accessors, and how polymorphism plays out across a typed class hierarchy with the compiler checking every override along the way.

1. Class Fields and Constructors

A TypeScript class declares its fields with a type annotation (either inline with an initializer, or declared and assigned in the constructor), and the constructor runs once when a new instance is created via `new`. Fields declared without an initializer and without being assigned in every constructor path produce a compile error under strict mode (strictPropertyInitialization) — this is a real safety feature, not noise, since it catches a field that could otherwise end up undefined despite its type claiming otherwise.

TYPESCRIPT

```
class Rectangle {
  width: number;
  height: number;

  constructor(width: number, height: number) {
    this.width = width;
    this.height = height;
  }

  area(): number {
    return this.width * this.height;
  }
}

const rect = new Rectangle(4, 5);
console.log(rect.area()); // 20
```

2. Access Modifiers: public, private, protected

public (the default, if omitted) members are accessible from anywhere; private members are accessible only within the declaring class itself, not subclasses or external code; protected members are accessible within the declaring class and any subclass, but not from outside the hierarchy entirely. These are compile-time-only checks, erased from the emitted JavaScript — unlike the newer, genuinely runtime-private JavaScript syntax (#field), TypeScript's private modifier can technically still be accessed by bypassing the type checker (bracket notation, a type assertion), so it's a strong convention enforced during development, not an unbreakable runtime guarantee.

- private is compile-time only; the native JavaScript `#field` syntax is truly private at runtime too, and TypeScript fully supports mixing both.
- protected is the right choice for members a subclass needs to use or override, but that external code should never touch directly.

```typescript
class BankAccount {
  private balance: number;
  protected readonly accountId: string;

  constructor(initialBalance: number, accountId: string) {
    this.balance = initialBalance;
    this.accountId = accountId;
  }

  deposit(amount: number): void {
    this.balance += amount;
  }

  getBalance(): number {
    return this.balance;
  }
}

const acc = new BankAccount(100, "ACC-1");
acc.deposit(50);
// acc.balance;   // error: 'balance' is private and only accessible within BankAccount
```

3. Inheritance, super, and Method Overriding

A class extends another with the extends keyword, and super(...) must be called inside a derived class's constructor before accessing `this`, forwarding arguments to the parent constructor. An overriding method can call super.methodName() to invoke the parent's version of that method as part of its own implementation, rather than fully replacing it — exactly the same pattern as Python's super(), just with different syntax.

```typescript
class Animal {
  constructor(protected name: string) {}

  speak(): string {
    return `${this.name} makes a sound`;
  }
}

class Dog extends Animal {
  speak(): string {
    return `${super.speak()}, specifically a bark`;
  }
}

console.log(new Dog("Rex").speak()); // "Rex makes a sound, specifically a bark"
```

4. Abstract Classes

An abstract class (declared with the abstract keyword) cannot be instantiated directly with `new` — it exists purely to be extended, and can declare abstract methods (abstract methodName(): ReturnType;) that have no implementation at all in the base class, forcing every concrete subclass to provide one. This is TypeScript's equivalent of Python's ABC pattern, and is the right tool when a base class should supply some shared implementation while still requiring subclasses to fill in specific pieces.

```typescript
abstract class Shape {
  abstract area(): number;

  describe(): string {
    return `This shape has an area of ${this.area()}`;
  }
}

class Circle extends Shape {
  constructor(private radius: number) {
    super();
  }

  area(): number {
    return Math.PI * this.radius ** 2;
  }
}

// new Shape();          // error: cannot create an instance of an abstract class
console.log(new Circle(2).describe());
```

5. Implementing Interfaces

A class declares that it satisfies an interface's shape with the implements keyword, and the compiler checks at the class declaration itself that every required member is actually present with a compatible type — catching a missing or mistyped method immediately, rather than only when some calling code tries to use it. Unlike extends (which provides actual inherited implementation), implements is a pure type-checking contract: a class can implement multiple interfaces, since there's no implementation to conflict between them.

```typescript
interface Flyable {
  fly(): string;
}

interface Swimmable {
  swim(): string;
}

class Duck implements Flyable, Swimmable {
  fly(): string {
    return "Duck is flying";
  }
  swim(): string {
    return "Duck is swimming";
  }
}
```

6. Static Members and Parameter Properties

static members belong to the class itself rather than any instance, accessed via ClassName.member rather than an instance — useful for factory methods, shared counters, or constants related to the class as a whole. Parameter properties are a TypeScript-only shorthand that collapses declaring a field and assigning it in the constructor into one line: prefixing a constructor parameter with public/private/protected/readonly simultaneously declares that field and assigns the passed-in value to it, removing a very common source of constructor boilerplate.

```typescript
class Counter {
  static instanceCount = 0;

  constructor(private readonly label: string) {
    // `private readonly label` above both declares AND assigns the field —
    // equivalent to declaring `private readonly label: string;` and then
    // `this.label = label;` inside the constructor body.
    Counter.instanceCount++;
  }

  describe(): string {
    return `${this.label} (#${Counter.instanceCount})`;
  }
}
```

```
new Counter("a");
new Counter("b");
console.log(Counter.instanceCount); // 2
```

7. Getters, Setters, and Polymorphism

get and set accessor methods let a class expose computed or validated values with plain property syntax (obj.value instead of obj.getValue()), mirroring Python's @property pattern. Combined with a typed class hierarchy, polymorphism in TypeScript means a variable typed as a base class or interface can hold any subclass/implementing-class instance, and calling an overridden method dispatches to that specific instance's implementation — all fully type-checked, so a method call that doesn't exist on the declared base type is caught at compile time rather than surfacing as a runtime error.

```typescript
class Temperature {
  private _celsius = 0;

  get fahrenheit(): number {
    return this._celsius * 9 / 5 + 32;
  }

  set fahrenheit(value: number) {
    this._celsius = (value - 32) * 5 / 9;
  }
}

const t = new Temperature();
t.fahrenheit = 98.6;
console.log(t.fahrenheit.toFixed(1)); // 98.6 — reads/writes like a plain property
```

Common Pitfalls

- Forgetting to call super(...) before accessing `this` inside a derived class's constructor, which the compiler flags as an error.
- Treating TypeScript's `private` as a genuine runtime guarantee — it's compile-time only and can be bypassed with a type assertion or bracket access.
- Trying to instantiate an abstract class directly with `new`, or forgetting to implement one of its abstract methods in a concrete subclass.
- Confusing `implements` (a pure type-checking contract, no shared implementation) with `extends` (actual inherited implementation) — a class can implement many interfaces but extend only one class.
- Writing out full field-declare-then-assign boilerplate in a constructor when the parameter-property shorthand (private readonly x: T) would do both in one line.
- Defining a setter without a matching getter (or vice versa) when the intent was a fully computed, two-way property.

Key Takeaways

- public/private/protected are compile-time-only access checks; use the native `#field` syntax if genuine runtime privacy is required.
- super(...) must run before `this` is used in a derived class's constructor, and super.method() calls the parent's version of an overridden method.
- Abstract classes can mix shared implementation with methods that force every subclass to provide their own — TypeScript's equivalent of Python's ABC.
- implements checks that a class satisfies an interface's shape at declaration time; a class can implement multiple interfaces but extend only one class.
- Parameter properties (constructor(private x: T)) collapse field declaration and assignment into the constructor's parameter list.
- get/set accessors expose computed or validated values through ordinary property syntax, enabling the same pattern as Python's @property.

Learn More

TypeScript Handbook — Classes
https://www.typescriptlang.org/docs/handbook/2/classes.html

TypeScript Handbook — Interfaces (implements)
https://www.typescriptlang.org/docs/handbook/2/objects.html

MDN — Classes (Private class features)
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Classes/Private_properties